

RiskTrace Engine Project Summary

RiskTrace Engine is a GenAI-assisted project-risk analysis system for software issue trackers. It ingests Jira-style issue data, builds a temporal dependency graph, propagates delay/blocker risk downstream, and answers natural-language questions about risk cascades. The problem is important because project delays rarely stay local: one blocked upstream task can quietly endanger many dependent tasks before a manager sees the pattern.

GenAI is justified here because the user task is not only scoring. A deterministic graph engine computes risk scores, while semantic retrieval and an LLM explanation layer translate graph evidence into natural-language root-cause summaries and mitigation recommendations. This is stronger than a traditional dashboard because the system can answer intent-level questions such as "which blocked tasks are causing downstream risk?" and retrieve relevant issues even when the wording differs.

Architecture at a glance: `PerceptionAgent` loads nodes/edges from CSV or Neo4j; `GraphReasoningAgent` runs multi-hop temporal risk propagation; `RAGRetriever` indexes issue documents with ChromaDB plus `all-MiniLM-L6-v2` and falls back to keyword matching when dependencies are unavailable; `PlanningAgent` ranks mitigations; `DecisionAgent` uses Groq LLaMA 3.3 70B for explanations; `CriticAgent` filters unsupported LLM output.

The standout contribution is counterfactual what-if analysis: resolving an issue is simulated in an in-memory clone, risk is recomputed, and the system reports before/after downstream risk deltas without writing to the database. This gives an interventional view of software dependency graphs rather than only a descriptive ranking.

Evaluation results in `evaluation/results.json` show the RAG pipeline outperforming baselines overall: ours achieves precision 0.889, recall 0.736, and F1 0.747, compared with keyword F1 0.548 and Cypher baseline F1 0.014. The included real-Hadoop sample contains 1,500 parser-visible issue records, 26 explicit dependency edges, and 4,327 inferred edges. Inferred edges were checked with a 100-edge heuristic validation sample, producing precision 0.500, recall 1.000, and F1 0.667; because those labels are heuristic rather than hand-adjudicated ground truth, dependency validation remains the main limitation. The prototype is designed for graceful degradation: CSV mode works without Neo4j, RAG falls back to keyword matching when model files are unavailable, and structured risk outputs still appear if the Groq LLM call cannot be made.